

Hitta ordningen

- **Tidsgräns:** 10 minuter
- **Miljö:** en GPU (≈ 16 GB VRAM), ingen internetuppkoppling
- **Lösningens storlek:** `solution.ipynb` ≤ 1 MB
- **Lagring:** 5 GB

Uppgift

Du får talade engelska dialoger mellan två deltagare, *Speaker A* och *Speaker B*. Varje dialog är uppdelad i talarturer, där varje tur innehåller tal från endast en talare. Varje tur lagras som en separat `.wav`-ljudfil, så en komplett dialog representeras av en mängd `.wav`-filer, en för varje tur.

Olyckligtvis har turerna blandats slumpmässigt, så samtalet är inte längre begripligt. I filnamnet `chunk_{k}.wav` syftar `k` på det `k`:te segmentet i den blandade mängden, inte på den `k`:te turen i den ursprungliga dialogen.

!! Din uppgift är att rekonstruera samtalets ursprungliga kronologiska ordning.



Dataset

Varje dialog innehåller n ljudfiler med namnen `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. Segmenten är enskilda turer. Filnamnen motsvarar endast den blandade ordningen. De anger inte var ett segment hör hemma i det ursprungliga samtalet. Varje dialog har 7–20 segment, mono, 44.1 kHz (du får omsampla).

`prefix.json` innehåller filnamnsindexen för de två första segmenten i varje dialog. Detta identifierar dialogens verkliga början och tar bort tvetydigheten mellan att läsa samtalet framåt eller bakåt.

Till exempel: 11: [7, 12] betyder att den första och andra turen i dialog 11 är `chunk_7.wav` respektive `chunk_12.wav`.

Vad du får

Du får **två mappar i identiskt format**:

Mapp	Dialoger	<code>answers.json</code> ?	Använd den för att
<code>dataset/train/</code>	1,288	☒ inkluderad	träna / finjustera din modell
<code>dataset/test_public/</code>	100	☒ inkluderad	köra din pipeline och poängsätta dig själv lokalt

Vid rättningen ersätts din mapp `dataset/test_public/` transparent av en hidden evaluation set (`test_leaderboard_a` för den offentliga topplistan och `test_leaderboard_b` för den slutliga topplistan) — dessa har samma storlek och format som `dataset/test_public/` men utan `answers.json`.

Din notebook körs igen på dessa data, och den `answers.json`-fil den producerar används för poängsättning. De undanhållna testdialogerna kommer från samma fördelning som `train`, så din lokala `test_public`-poäng är en trogen förhandsvisning.

Katalogstruktur

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

Utdata

Bestäm för varje dialog den ursprungliga kronologiska ordningen för dess ljudsegment. Din prediktion ska vara en permutation P av $\{0, 1, \dots, n-1\}$, där $P[i]$ är den predikterade kronologiska positionen för `chunk_i.wav` (0 = först).

Din utdatafil `answers.json` ska mappa varje dialog-ID till dess predikterade permutation:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

Exempel

En dialog har 3 blandade segment `chunk_0`, `chunk_1`, `chunk_2`:

blandat segment	talat innehåll	sann position (rang)
<code>chunk_0.wav</code>	<i>"No worries — I'll send you the notes afterwards."</i>	2 (sist)
<code>chunk_1.wav</code>	<i>"Hey, are you coming to the three o'clock meeting?"</i>	0 (först)
<code>chunk_2.wav</code>	<i>"I can't — I've got a dentist appointment then."</i>	1

Den sanna ordningen är **chunk_1** → **chunk_2** → **chunk_0**, så $P = [2, 0, 1]$, och `prefix.json` innehåller $[1, 2]$.

⚠ **P måste vara en genuin permutation:** längd n , 0-indexerad, varje värde exakt en gång. Dubletter, saknade värden eller poster utanför intervallet (t.ex. 1-indexerade) ger 0 poäng för den dialogen, liksom en dialog som saknas i filen. En felformad fil eller en fil som inte är JSON avisas.

Poängsättning

Poängsättningen för denna uppgift är **parvis ordningsnoggrannhet** (pairwise ordering accuracy). Den kontrollerar varje par av segment och frågar: *vilket av de två ska komma först?* Ett par är korrekt om din prediktion ger samma svar som sanningsvärdet (ground truth). För en dialog med n segment finns det

$$M = n(n - 1)/2$$

par; låt I vara antalet inversioner — par som är ordnade annorlunda än i ground truth:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

i **Slutpoängen** är **genomsnittet av poängen per dialog över alla dialoger i splitten**.

Tillåtna modeller

Du får endast använda följande förtränade modeller för att lösa denna uppgift, både under träning och utvärdering. Alla dessa modeller är redan nedladdade och tillgängliga i miljön. Du kan se exempel på hur du använder dem i baseline-notebooken `solution.ipynb`. Observera att du inte får använda någon annan modell, och att ditt program inte har internetåtkomst.

- **Talrepresentationer: wav2vec 2.0. Whisper-encodern** får också användas som feature extractor. [wav2vec model card](#)
- **Automatisk taligenkänning (ASR): OpenAI Whisper** (valfri storlek). [Whisper model card](#)
- **Språkmodell: Qwen2.5-0.5B**, som får användas antingen zero-shot eller finjusterad på den tillhandahållna `train`-splitten. [Qwen model card](#) Observera att tidsgränsen på 10 minuter måste täcka all träning eller finjustering du gör vid rättningstillfället plus inferens på utvärderingsmängden.

Hur du skickar in

- Öppna `solution.ipynb` och kör alla celler. Kontrollera att den skriver `answers.json` i arbetskatalogen med en permutation för varje dialog i `dataset/test_public/` (100

dialoger). Vid rättningstillfället körs notebooken om på den dolda testmängden och den `answers.json` som produceras där poängsätts.

- Förbättra lösningen om du vill — eller låt bli; baseline-lösningen i sig validerar pipelinen.
- Öppna Git-fliken i vänstra sidofältet i JupyterLab.
- **Stage:** `solution.ipynb` (+-ikonen intill den).
- Skriv in ett commit-meddelande och klicka på **Commit**.
- Klicka på molnet med uppåtpil för att pusha.
- Återvänd till denna Contest-sida och klicka på **Submit**.

Skicka in exakt en fil, med namnet `solution.ipynb`.